

Microcontroladores

Roteiro 2 — Laços, decisão e leitura de entradas

Raoni F. S. Teixeira · DENE/UFMT · 1 sessão · 1,0 ponto · grupos de 3

Objetivos desta sessão

Ao final desta sessão o grupo deve ser capaz de:

1. Escrever laços `for` e `while` em C aplicados a saída física;
2. Explicar por que o programa embarcado termina em laço infinito;
3. Ler chaves e botões usando resistores de *pull-up* internos;
4. Reconhecer e descrever o repique de contato mecânico.

1 Configuração de bancada

Configuração de bancada

Chaves SWITCHS (PORTB)	Todas em OFF no início
CH2-1 (LCD)	OFF
CH3-3 e CH3-5	OFF

As chaves SWITCHS serão usadas na Parte 3. Até lá, mantenha-as em OFF — quando ligadas, elas aterram os pinos do PORTB.

2 Parte 1 — O laço infinito

2.1 O programa que termina

Tarefa

Grave este código e observe:

```
void main(void)
{
    TRISD = 0x00;
    LATD  = 0x00;    /* acende todos */
}
```

1.1 Os LEDs acendem? Permanecem acesos? O que acontece depois que `main` termina?

O comportamento observado costuma surpreender. Anote exatamente o que viu, inclusive se for intermitente.

2.2 A forma normal

```
void main(void)
{
    TRISD = 0x00;

    for (;;) {
        LATD = 0x00;
        __delay_ms(300);
        LATD = 0xFF;
        __delay_ms(300);
    }
}
```

- 1.2 Em um sistema embarcado não existe sistema operacional para o qual retornar. Se main terminasse, para onde o processador iria?

- 1.3 Compare com um programa de Algoritmos que você escreveu antes. Onde estava o “fim” naquele programa? Por que aqui não há?

Conceito central

Em um computador de propósito geral, o programa é hóspede: nasce, executa e devolve o controle ao sistema operacional. Em um microcontrolador, o programa **é** o sistema. Não há para onde voltar, e por isso o laço infinito não é um recurso — é a estrutura obrigatória.

3 Parte 2 — Laços com contador

3.1 Sequência

Tarefa

2.1 — Escreva um programa que acenda os LEDs **um de cada vez**, do 0 ao 7, e recomece. Use um for e a construção de máscara do Roteiro 1.

```
for (;;) {
    for (uint8_t i = 0; i < 8; i++) {
        LATD = ~(uint8_t)(1u << i);    /* por que o ~ ? */
        __delay_ms(150);
    }
}
```

2.2 Explique a linha marcada. Por que `1u << i` precisa ser invertido antes de ir para LATD?

3.2 Contador binário

Tarefa

2.3 — Modifique o programa para que os LEDs mostrem uma **contagem binária de 0 a 255**, incrementando a cada 200 ms.

🔧 Previsão — registre ANTES de gravar

2.4 — Quanto tempo leva um ciclo completo da contagem?

Calcule **antes** de medir. Depois confira com um cronômetro e compare.

Previsto: _____ Medido: _____

Se houver divergência maior que 10%, o que pode explicá-la?

3.3 Ida e volta

Tarefa

2.5 — Modifique a sequência do item 2.1 para ir do LED 0 ao 7 e **voltar** do 7 ao 0, sem repetir os extremos.

Dica: dois laços, ou um laço com variável de direção. Escolha uma abordagem e justifique em uma frase.

4 Parte 3 — Entradas

4.1 Fundamento

As 8 chaves DIP da seção SWITCHS estão ligadas ao PORTB e são **ativas em nível baixo**: em ON elas aterram o pino.

Quando abertas, quem garante o nível alto são os **resistores de pull-up internos** do PORTB, que precisam ser habilitados por software:

```
INTCON2bits.RBPU = 0; /* 0 = pull-ups habilitados */
```

3.1 O bit se chama RBPU — *PORTB Pull-Up disable*. Por que escrever **zero** habilita os resistores?

4.2 Espelho

🔧 Previsão — registre ANTES de gravar

3.2 — Considere o programa abaixo, que pretende refletir o estado das chaves nos LEDs.

```
TRISB = 0xFF;           /* PORTB como entrada */
TRISD = 0x00;
INTCON2bits.RBPU = 0;

for (;;) {
    LATD = PORTB;
}
```

Ele faz o que se pede? Considere: a chave em ON produz nível **baixo**, e o LED aceso exige nível **baixo**.

Preveja o comportamento antes de gravar.

Tarefa

3.3 — Grave, verifique e corrija se necessário. O resultado correto é: chave ligada → LED aceso.

4.3 Contagem de acionamentos

Tarefa

3.4 — Use o push-button **INT0 (SW12, em RB0)**. Escreva um programa que incremente um contador a cada aperto e mostre o valor em binário nos LEDs.

3.5 Aperte o botão dez vezes, devagar. O contador marca dez? Anote o valor obtido em três tentativas.

Observação

Se marcou mais que dez, você encontrou o **repique de contato**: a lâmina metálica do botão bate várias vezes antes de estabilizar, e o programa lê cada batida como um acionamento novo.

Isso não é defeito do kit nem do seu código. É o comportamento físico de qualquer contato mecânico, de um botão de campainha a um contator de potência.

Tarefa

3.6 — Proponha uma solução para o repique, em português, sem escrever código.

O tratamento formal virá no Roteiro 7, quando vocês tiverem uma base de tempo periódica disponível — a ferramenta que falta hoje.

5 Teste de aceitação

- ☐ Sequência de ida e volta funcionando (item 2.5).
- ☐ Espelho das chaves nos LEDs, com a polaridade correta (item 3.3).
- ☐ Contador de acionamentos funcionando, mesmo que com repique.

6 Entrega e critério

Peso	Item
0,2	Teste de aceitação aprovado
0,2	Respostas 1.2 e 1.3 sobre o laço infinito
0,2	Resposta 2.2 e cálculo 2.4 confirmado por medição
0,2	Respostas 3.1 e previsão 3.2
0,2	Proposta 3.6 para o repique

Tabela 1: Distribuição do ponto do Roteiro 2.

7 Armadilhas frequentes

Sintoma	Causa provável
LEDs invertidos no espelho	Faltou o ~: as duas polaridades se somam
Chaves sem efeito	<i>Pull-ups</i> não habilitados, ou TRISB errado
Contador dispara sozinho	Repique — esperado nesta etapa
Programa “trava” após um instante	main sem laço infinito
Tempos errados por fator fixo	_XTAL_FREQ diferente do clock real: use 16 MHz

Tabela 2: Diagnóstico rápido do Roteiro 2.

8 Para a próxima sessão

Tarefa

Até agora a saída são oito LEDs, que mostram um byte.

Pense e traga escrito: como você faria para mostrar o número **decimal** 37? E a palavra TEMP? O que seria necessário além do que você já tem?